

VS CODE DEVELOPER WORKSPACE MASTER INTEGRATION PACK

Core Project Root: school-monolith-core

This directory package acts as your unified engineering directive inside Visual Studio Code. It provides the exact system specifications, metadata structures, configurations, and baseline operations to implement Phase 1—incorporating the core AI/WhatsApp pipelines along with **Fee Management**, **Attendance Frameworks**, and **Automated Timetable Planners**.



VS Code Workspace Configuration Environment

To ensure type checking, strict linting-on-save, and automated code formatting across your team, place these configuration files inside a `.vscode/` directory at your root folder.

`.vscode/settings.json`

```
JSON
{
  "editor.formatOnSave": true,
  "editor.codeActionsOnSave": {
    "source.fixAll.eslint": "always"
  },
  "editor.defaultFormatter": "esbenp.prettier-vscode",
  "typescript.tsdk": "./node_modules/typescript/lib",
  "typescript.enablePromptUseWorkspaceTsdk": true,
  "files.associations": {
    "*.sql": "postgres"
  },
  "search.exclude": {
    "**/node_modules": true,
    "**/dist": true,
    "**/.next": true
  },
  "editor.rulers": [80, 120]
}
```

.vscode/launch.json

```
JSON
{
  "version": "0.2.0",
  "configurations": [
    {
      "type": "node",
      "request": "launch",
      "name": "Launch Monolith: Debug API Engine",
      "runtimeArgs": "-r" "ts-node/register",
      "args": "src/main.ts",
      "envFile": "${workspaceFolder}/.env",
      "skipFiles": "<node_internals>/**",
      "outFiles": "${workspaceFolder}/dist/**/*.js"
    }
  ]
}
```

.vscode/tasks.json

```
JSON
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Database: Execute Migrations",
      "type": "shell",
      "command": "npm run db:migrate",
      "problemMatcher": [],
      "group": "build"
    }
  ]
}
```

```
"label" "Docker: Spin Up Local Stack"
"type" "shell"
"command" "docker-compose up -d redis postgres"
"problemMatcher"
```

1. Product Requirement Document (PRD)

1.1 Document Overview & Objective

The platform acts as an administrative assistant ecosystem tailored to Indian private K-12 institutions. By targeting the intersection of operations and daily routines, the app provides a highly visual interface for school leaders, automated preparation toolkits for educators, and a friction-free WhatsApp channel for families.

1.2 Enhanced Functional Modules

FR-1: 1-Click Central Dashboard

- **Requirement:** Complete data synthesis across modules into a single, high-speed student profile view.
- **Capability:** Clicking a profile pulls up instant records for tracking academic progression, non-scholastic trends, financial status balances, and rolling attendance metrics without requiring multi-department callbacks.

FR-2: 360-Degree Progress & Evaluation Engine

- **Requirement:** Continuous logging of non-scholastic parameters mapping to the competency frameworks of the National Education Policy (NEP 2020).
- **Capability:** Provides simple 1-5 slider rubrics for soft-skill tracking (Critical Thinking, Reading Skills, Teamwork, Comprehension). Generates clean, client-side Spider-Web charts for parent reviews.

FR-3: GenAI Teacher Assistant Toolkit

- **Requirement:** Automatic creation of classroom lesson resources.
- **Capability:** On-demand generation of structured lesson timelines, summaries, visual mind map maps, and active recall flashcard structures.
- **Active Recall Pipeline:** Generates three automated daily quiz items from the active teaching units to power continuous **weakness tracking loops**.

FR-4: WhatsApp Business Automation Layer

- **Requirement:** Completely removes app onboarding barriers for guardians.
- **Capability:** Outbound webhook handling for tracking daily student check-ins, delivering broadcast updates, and processing text-based report requests (e.g., sending "REPORT" immediately yields the latest generated 360° progress card PDF).

FR-5: Fee Management & Reconciliation Module

- **Requirement:** Handles multi-tiered fee payment structures, cash tracking, and electronic payments.
- **Capability:** Supports customized fee structures across class groups (e.g., sibling discounts, scholarship provisions). Features clear collection tracking with automatic notification loops via WhatsApp to flag late installments before billing dates.

FR-6: Attendance Tracker Framework

- **Requirement:** High-speed daily logging across classes.
- **Capability:** Web-based student ledger lists for fast teacher inputs. Tracks historical leaves and interfaces natively with physical biometric systems or RFID scanning lines.

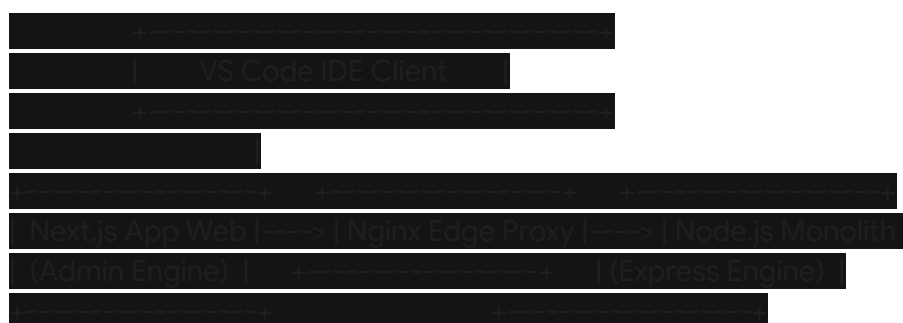
FR-7: Timetable Planner System

- **Requirement:** Generates conflict-free scheduling blocks across rooms and resources.
- **Capability:** Grid matrix layout rules tracking teacher availability hours, active rooms, and section splits. Flags structural scheduling overlaps to ensure error-free operational calendars.

2. System Architecture Document (SAD)

2.1 Complete System Topology

A **Modular Monolith** pattern keeps development workflows simple while isolating logical processing loops inside strict code domains. High-latency pipelines (AI inference generation and WhatsApp message deliveries) are offloaded to asynchronous processing systems.




```

last_name VARCHAR 100 NOT NULL
grade_section VARCHAR 20 NOT NULL
guardian_phone VARCHAR 15 NOT NULL
metadata JSONB DEFAULT '{}' isonb -- Height, weight, health telemetry
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP

```

```

CREATE TABLE attendance
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
student_id UUID REFERENCES students(id) ON DELETE CASCADE
date DATE NOT NULL
status VARCHAR 20 NOT NULL -- 'PRESENT', 'ABSENT', 'LATE', 'LEAVE'
logged_by VARCHAR 100 NOT NULL
created_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
CONSTRAINT unique_student_date UNIQUE (student_id, date)

```

```

CREATE TABLE fee_categories
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
school_id UUID REFERENCES schools(id) ON DELETE CASCADE
name VARCHAR 100 NOT NULL -- 'Tuition Fee', 'Transport Fee', 'Lab Fee'
amount_in_cents BIGINT NOT NULL -- Stored in lowest currency unit to prevent float errors
due_date DATE NOT NULL

```

```

CREATE TABLE student_fee_ledgers
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
student_id UUID REFERENCES students(id) ON DELETE CASCADE
fee_category_id UUID REFERENCES fee_categories(id) ON DELETE CASCADE
amount_paid_in_cents BIGINT DEFAULT 0
discount_in_cents BIGINT DEFAULT 0
status VARCHAR 30 DEFAULT 'UNPAID' -- 'UNPAID', 'PARTIAL', 'PAID'
updated_at TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP

```

```

CREATE TABLE timetable_slots
id UUID PRIMARY KEY DEFAULT gen_random_uuid()
school_id UUID REFERENCES schools(id) ON DELETE CASCADE
grade_section VARCHAR 20 NOT NULL
day_of_week INT NOT NULL -- 1 = Monday, 5 = Friday
start_time TIME NOT NULL
end_time TIME NOT NULL
subject VARCHAR 100 NOT NULL

```

```

teacher_id VARCHAR 100 NOT NULL
room_number VARCHAR 50 NOT NULL
CONSTRAINT unique_schedule_overlap UNIQUE (school_id, day_of_week, start_time,
room_number)

-- Optimization Indexing For Fast 1-Click Profiling
CREATE INDEX idx_student_lookup_main ON students (school_id, grade, section)
CREATE INDEX idx_attendance_date_search ON attendance (date, status)
CREATE INDEX idx_fee_ledger_status ON student_fee_ledgers (student_id, status)

```

3.2 Principal Core System Endpoints

GET /api/v1/operations/timetable/verify

- **Description:** Verifies timetable slots against scheduling overlap parameters.
- **Response Payload Output:**

```

JSON
{
  "isValid": false,
  "conflictsDetected": {
    "type": "TEACHER_OVERLAP",
    "message": "Teacher [ID: T-102] is already assigned to Section 6-A in Room 204 during 09:00:00 - 09:45:00.",
    "violatingSlot": { "day": 1, "time": "09:00:00" }
  }
}

```

POST /api/v1/finance/ledger/reconcile

- **Description:** Logs incoming payments and updates student ledger states.
- **Payload Specification:**

JSON

```
{
  "studentId" "b6a71e89-1229-4bf9-9941-7c9632aee142"
  "feeCategoryId" "fee-88214-cb91"
  "paymentAmountCents" 1500000
  "paymentMode" "UPI"
  "referenceNumber" "UPI442910482"
}
```

4. Engineering Specification (Spec)

4.1 System Development Stack Constraints

- **Runtime Runtime Engine:** Node.js, running with strict TypeScript execution layers.
- **Data Tier Layers:** PostgreSQL utilizing core connection pooling rules. Redis manages active server cache operations.
- **Validation Policies:** Network payloads must use runtime validation engines (Zod schemas) before hitting database lines.

4.2 Algorithm: Timetable Conflict Matrix Intersect Check

To ensure schedule validity, scheduling matrices evaluate cross-product interactions across three primary constraints: Teacher Allocation, Room Capacity, and Student Batch Tracks. Let an active timetable slot instance be defined as a tuple profile element SS :

$$SS = (T, R, B, D, [t_{\text{start}}, t_{\text{end}}])$$

Where T is the unique Teacher assignment key, R is the localized Room assignment number, B maps the Target Student Batch, D is the Target Day identifier index, and $[t_{\text{start}}, t_{\text{end}}]$ is the active processing time window block.

A conflict flag condition evaluates as true if any two slot schedules (S_1 and S_2) share identical space-time tracking domains:

$$\text{Conflict}(S_1, S_2) \iff (D_1 = D_2) \wedge ([t_{\text{start}1}, t_{\text{end}1}] \cap [t_{\text{start}2}, t_{\text{end}2}] \neq \emptyset) \wedge \big((T_1 = T_2) \vee (R_1 = R_2) \vee (B_1 = B_2)\big)$$

TypeScript

```
// Timetable conflict validation logic mapping block
export interface TimeSlot {
  teacherId: string
  roomNumber: string
  batchId: string
  dayOfTheWeek: number
  startTime: number // Stored as minutes from midnight (e.g., 540 = 09:00)
  endTime: number
}

export function evaluateScheduleConflict(slot1: TimeSlot, slot2: TimeSlot): boolean {
  if (slot1.dayOfTheWeek !== slot2.dayOfTheWeek) return false

  // Evaluate operational intersection conditions
  const hasTimeOverlap = Math.max(slot1.startTime, slot2.startTime) <
    Math.min(slot1.endTime, slot2.endTime)
  if (!hasTimeOverlap) return false

  // Check individual constraint collisions
  const teacherCollision = slot1.teacherId === slot2.teacherId
  const roomCollision = slot1.roomNumber === slot2.roomNumber
  const batchCollision = slot1.batchId === slot2.batchId

  return teacherCollision || roomCollision || batchCollision
}
```

5. Standard Operating Procedure (SOP)

5.1 SOP-101: Initial Dev Local Sandbox Onboarding inside VS Code

1. **Clone Project Assets:** Clone the core source repository from your source repository down to your local engineering development directory.
2. **Open Workspace Container:** Fire up VS Code and open the core repository directory. When prompted at the bottom right, select "**Use Workspace TypeScript Version**" to inherit the shared compiler configuration rules.
3. **Install Target Extensions:** Install the recommended workspace tooling engines via the command palette (Ctrl+Shift+P -> Extensions: Show Recommended Extensions):
 - *ESLint* (dbaeumer.vscode-eslint)
 - *Prettier* (esbenp.prettier-vscode)
 - *PostgreSQL Engine Client* (ckolkman.vscode-postgres)
4. **Spin Up Local Services Stack:** Launch the integrated VS Code terminal instance

(`Ctrl+``) and execute the configuration runner script to spin up background infrastructure resources:
npm run stack:local

5. ****Initialize Environment Variables:**** Duplicate the distributed `.env.example` template to generate an active `.env` configuration file. Ensure local connection ports align perfectly with your container endpoints.

6. ****Run Migrations:**** Open the Command Palette (`Ctrl+Shift+P`), run `Tasks: Run Task` and choose ****Database: Execute Migrations**** to build out target schema layouts.

7. ****Launch Debug Lifecycle Engine:**** Hit `F5` on your keyboard. This matches the settings mapped in `.vscode/launch.json` to start up the backend server, placing active break-points safely into the runtime engine code.